

Lecture 5: Distance

How far apart are two countries? — six honest answers, and what each one throws away

STAT 481 — Multivariate Analysis

Thu, Sep 3, 2026 • 50 minutes

Today's Overview

1. **The data today.** `worldhealth2022` — the same 175 countries and eight indicators as Tuesday, with the country names on the rows. The wider file, `whwide2022`, lends one column in §1 and carries the home problems.
2. **What a distance is.** The definition, one distance *proved* to deserve the name, and the first computations in R: the countries nearest to Qatar and farthest from it — on one column, then two, then three.
3. **One ordinary question with no single answer.** *How do we find the countries that are like Qatar? By which criteria? Which similarities should count?*
4. **Several ways to measure similarity between observations — six of them. Which one do we use?** We run all six on the same data and read the answers side by side, with Qatar in the front row.
5. **Two of them head to head, then a recipe.** Are the two main rivals the same or different — and when can they agree? Then a step-by-step workflow for choosing, a warm-up, home problems on the wider data, and an FAQ.

Learning Objectives

By the end of this lecture you should be able to:

1. state what a distance is, and verify the four rules for the straight-line distance;
2. build every pairwise distance between the rows of a data table with `dist()`, and read a heat map of the result;
3. explain the difference between the Euclidean and the Mahalanobis distance, and state exactly when the two return the same answers;
4. follow the workflow of §4: given a new table, decide which distance fits which purpose, and defend the choice in two sentences.

Packages today

- `pheatmap` [NEW] — `pheatmap()`, which draws a distance matrix as a colour-coded table (a *heat map*).

Load it in your *first* chunk. Open `STAT481.Rproj` first, so `"data/worldhealth2022.csv"` resolves. Everything else today is base R. Indicators: World Bank WDI, 2022.

1 What a distance is

In this section

The question. *What does “distance” mean for two countries — and does the distance from school deserve the name?*

What we do about it. Define it. Prove that the straight-line distance obeys all four rules. Then compute it in R — on one column, on two columns in the same unit, and on three columns in mixed units — and watch the answer stay sensible twice and break the third time. Close with the catalogue of six distances.

The definition

Definition 1.1: Distance

A **distance** is a rule d that attaches **one number** $d(P, Q)$ to every pair of units P, Q . Four rules, and nothing else:

- **The order does not matter** (symmetry): Qatar to Oman is Oman to Qatar.

$$d(P, Q) = d(Q, P)$$

- **Zero exactly for identical units:** every column equal — then, and only then, the distance is 0.

$$d(P, Q) = 0 \iff P = Q$$

- **Otherwise positive:** two different units are some positive distance apart, never a negative one.

$$d(P, Q) > 0 \text{ whenever } P \neq Q$$

- **No shortcut through a third unit** (the **triangle inequality**): a detour is never shorter.

$$d(P, Q) \leq d(P, R) + d(R, Q)$$

The first example — and a proof it deserves the name

Fact 1.1: The Euclidean (straight-line) distance is a distance

For two units $\mathbf{x}, \mathbf{y}$ measured on p columns,

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^p (x_k - y_k)^2}.$$

Check the four rules:

- *Symmetry:* $(x_k - y_k)^2 = (y_k - x_k)^2$ — squaring forgets the sign.
- *Zero exactly for identical units:* a sum of squares is 0 only when **every** term is 0, that is $x_k = y_k$ for every k .
- *Otherwise positive:* if any one column differs, one square is already > 0 , so the whole root is.
- *Triangle inequality:* put $\mathbf{u} = \mathbf{x} - \mathbf{z}$ and $\mathbf{v} = \mathbf{z} - \mathbf{y}$, so $\mathbf{x} - \mathbf{y} = \mathbf{u} + \mathbf{v}$. Squaring the claim $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ and cancelling the common terms leaves

$$\sum_{k=1}^p u_k v_k \leq \sqrt{\sum_k u_k^2} \sqrt{\sum_k v_k^2},$$

which is the **Cauchy–Schwarz inequality** — the same fact that kept every correlation in Lecture 2 between -1 and 1 . $\square$

Now in R: one column, then two, then three

First, today's file — same as Lecture 3, with the country *names* on the rows, so every ranking below reads:

```
library(pheatmap) # draws a distance matrix as a heat map
wh <- read.csv("data/worldhealth2022.csv", row.names = 1) # Lecture 2
X <- as.matrix(wh[, 4:11]) # the eight indicators -- Lecture 3's X
rownames(X) <- wh$country # country NAMES, so every ranking below reads
dim(X)
```

```
[1] 175 8
```

Question 1.1

Start with one column: *pop65*, the share of the population aged 65 and over. Which two countries are closest to Qatar, and which two are farthest?

On one column the Euclidean distance is just the gap with the sign dropped:

```
v <- X[, "pop65"] # ONE column: % of population aged 65+
d1 <- abs(v - v["Qatar"]) # the gap to Qatar, sign dropped
sort(d1)[2:3] # the two closest; no. 1 is Qatar itself
sort(d1, decreasing = TRUE)[1:2] # and the two farthest
```

United Arab Emirates	Zambia
0.249	0.465
Monaco Japan	
35.670 28.012	

Answer. Closest: the United Arab Emirates and Zambia; farthest: Monaco and Japan.

- `sort()` puts a vector in increasing order and the country names ride along; its argument `decreasing = TRUE` reverses the order. Position 1 of the increasing sort is Qatar itself, at distance 0 — hence `[2:3]`.
- The numbers read directly: the UAE's share of over-65s is within a quarter of a *percentage point* of Qatar's; Monaco's is 35.7 points away.
- Every answer here is sensible, and everyone agrees what the number means. One column is the easy case.

Question 1.2

Add a second column in the **same unit**: *pop0014*, the share aged 0–14 — also a percent of the population. Do the answers survive?

The extra column lives in the wider file, so we read it and check that the two files line up first:

```
w <- read.csv("data/whwide2022.csv", stringsAsFactors = FALSE)
all(w$country == wh$country) # same 175 rows, same order?
P2 <- cbind(pop65 = v, pop0014 = w$pop0014) # same UNIT: % of population
rownames(P2) <- wh$country
m2 <- as.matrix(dist(P2)) # every pairwise distance, as a table
sort(m2["Qatar", ])[2:3] # two closest
sort(m2["Qatar", ], decreasing = TRUE)[1] # and the farthest
```

```
[1] TRUE
United Arab Emirates Kuwait
1.252990 4.193299
Monaco
35.74713
```

Answer. Yes. Closest: the United Arab Emirates, then Kuwait; farthest: Monaco again.

- TRUE is doing real work: it says the two files hold the same 175 countries in the same order, so the columns can sit side by side.
- `dist(P2)` is new — it computes the distance between **every pair of rows** in one call; `as.matrix()` turns the result into a table indexed by name. §2 takes it slowly.
- Both columns are percentages of the same population, so adding them into one distance is fair — and the answer stays an age-structure answer: young Gulf populations close, old rich Monaco far.

Question 1.3

Now add a *third* column in a **different** unit: *gdp_pc*, income per person in US dollars. Same recipe — what happens?

```
P3 <- cbind(P2, gdp_pc = X[, "gdp_pc"])      # add DOLLARS to percentages
m3 <- as.matrix(dist(P3))
sort(m3["Qatar", ])[2:3]
round(P3[c("Qatar", "Singapore", "United Arab Emirates"), ], 2)
```

```
Singapore  Switzerland
2526.261    9107.650
                pop65 pop0014    gdp_pc
Qatar                1.39   15.25 88701.46
Singapore            12.69   12.01 91227.70
United Arab Emirates  1.64   16.48 50759.76
```

Answer. The answer breaks. Qatar's nearest neighbour is suddenly **Singapore** — where 12.7% of people are over 65 against Qatar's 1.4% — and the UAE, nearly identical to Qatar on both age columns, is gone from the list.

- Read the printed rows: on the two age columns Qatar and the UAE are near-twins; on income Qatar and Singapore are close.
- The distances are now in the thousands — they are essentially *dollar* gaps. A one-point age difference and a one-dollar income difference count the same in the sum, so the age columns stopped voting.
- **Same recipe, one extra column, wrong answer.** The rest of the lecture is what to do about exactly this.

Six distances: the catalogue

The straight line is not the only rule that satisfies Definition 1.1. Six are in common use. Three need nothing but the gaps between the two units; the others also want to know how the columns behave — for those, the worked pair is

$$\mathbf{x} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 0 \\ 5 \end{pmatrix}, \quad \mathbf{x} - \mathbf{y} = \begin{pmatrix} 3 \\ -4 \end{pmatrix}, \quad \mathbf{S} = \begin{pmatrix} 1 & 1 \\ 1 & 4 \end{pmatrix}$$

(**S**: variances $s_{11} = 1$, $s_{22} = 4$, covariance $s_{12} = 1$).

Name (synonyms)	Formula	Worked value
Euclidean <i>straight-line</i>	$\sqrt{\sum_{k=1}^p (x_k - y_k)^2}$	$\sqrt{3^2 + 4^2} = \sqrt{25} = \mathbf{5}$
City-block <i>Manhattan, taxicab</i>	$\sum_{k=1}^p x_k - y_k $	$3 + 4 = \mathbf{7}$
Chebyshev <i>largest-gap</i>	$\max_k x_k - y_k $	$\max(3, 4) = \mathbf{4}$
Standardised Euclidean	$\sqrt{\sum_{k=1}^p \frac{(x_k - y_k)^2}{s_{kk}}}$	$\sqrt{\frac{9}{1} + \frac{16}{4}} = \sqrt{13} = \mathbf{3.606}$
Correlation distance	$1 - r(\mathbf{x}, \mathbf{y})$, the correlation between the two units' <i>profiles</i> ; needs $p \geq 3$	on (1, 2, 3) vs (10, 20, 30): $1 - 1 = \mathbf{0}$
Mahalanobis <i>statistical</i>	$\sqrt{(\mathbf{x} - \mathbf{y})^\top \mathbf{S}^{-1} (\mathbf{x} - \mathbf{y})}$	$\sqrt{\frac{76}{3}} = \mathbf{5.033}$

Three notes on the table:

- **Correlation distance has its own tiny pair.** With only two columns, the correlation of two profiles is always ± 1 , so the row needs $p \geq 3$: on (1, 2, 3) against (10, 20, 30) — the same shape, ten times the level — it returns 0.
- **$\mathbf{S}^{-1}$ is the matrix that undoes $\mathbf{S}$;** R computes it, and what it is doing here becomes visible in §3. For our pair the number under the root is $[s_{22} \cdot 3^2 - 2s_{12} \cdot 3 \cdot (-4) + s_{11} \cdot (-4)^2] / (s_{11}s_{22} - s_{12}^2) = (36 + 24 + 16)/3 = 76/3$.
- **Which one to reach for, and when, is §4's workflow** — after §2 has run all six and §3 has taken the two main rivals apart.

On eight columns in different units these six do *not* agree, and the next section runs all of them.

2 Implementation in R: six distances, one question

In this section

The question. Which country is most like Qatar — and does the answer depend on which of the six distances we pick?

What we do about it. Build all six distance matrices with `dist()`, introduced slowly. For each one: the code, Qatar's three nearest neighbours, what the distance got right, and what mistake it made. Then draw all six matrices as heat maps on one page and read them side by side. **What the section leaves behind:** the raw material for §3's comparison and §4's workflow.

§1 already used `dist()` twice in passing. Here is the function properly, one argument at a time:

- `dist(x, method = "euclidean")` — $\mathbf{x}$ is a matrix whose **rows are the units**; the result holds the distance between every pair of rows, all $175 \times 174/2$ of them.
- `method =` picks the formula from §1's table: "euclidean" is the default; "manhattan" is the city-block row; "maximum" is the largest-gap row.
- `as.matrix()` turns the compact result into an ordinary 175×175 table — one row and one column per country, so `m["Qatar", ]` is Qatar's distance to everyone.
- `sort()` orders a named vector, keeping the names; `[2:4]` skips position 1, which is always the country itself at distance 0.

The six, one at a time

Question 2.1

Start with the straight line, now on all eight columns. Who is most like Qatar? Write one name down first.

```
D_euc <- dist(X)           # ALL pairwise straight-line distances
m_euc <- as.matrix(D_euc)  # the same thing as a 175 x 175 table
sort(m_euc["Qatar", ])[2:4] # nearest three; no. 1 is Qatar itself
```

Singapore	Switzerland	United States
2526.263	9107.667	10692.813

Answer. Singapore, then Switzerland, then the United States.

- **Right:** nothing to choose, nothing to tune — and on columns in one common unit (§1's P2) it was the honest answer.
- **The mistake:** the same one as P3, now with eight columns: it adds dollars to years to percentages. `gdp_pc` varies by hundreds of millions (variance 8.0×10^8) against `fertility`'s 1.6, so income decides alone — these are P3's numbers almost to the dollar.

Question 2.2

Maybe the squaring is the problem — one huge gap, squared, swamps everything. Add the plain gaps instead: does the answer change?

```
D_city <- dist(X, method = "manhattan")
m_city <- as.matrix(D_city)
sort(m_city["Qatar", ])[2:4]
```

Singapore	Switzerland	United States
2547.256	9154.379	10752.171

Answer. No. The same three names, in the same order.

- **Right:** a wild value in one column counts once, not squared — city-block is the robust sibling of the straight line.
- **The mistake:** it still adds dollars and percentages as if they were the same thing. Changing the formula repaired nothing, because the formula was never the problem.

Question 2.3

Go to the other extreme: score each pair only by its single worst gap.

```
D_cheb <- dist(X, method = "maximum")
m_cheb <- as.matrix(D_cheb)
sort(m_cheb["Qatar", ])[2:4]
```

Singapore	Switzerland	United States
2526.234	9107.633	10692.773

Answer. The same names again — and almost the same numbers: 2526.23 against the straight line's 2526.26.

- **Right:** “how bad is the worst gap?” is a real question — tolerance and quality checks ask exactly it.

- **The mistake:** on this table the worst gap is *always* the income gap, so largest-gap *is* the income distance in disguise. That the two lists agree to a fraction of a dollar is the proof that one column is doing all the talking.

Question 2.4

Put every column on its own scale first — mean 0, spread 1 — and only then take the straight line. Same recipe, fairer columns: who is Qatar's neighbour now?

```
Z <- scale(X) # every column: mean 0, sd 1 (Worksheet 3)
D_std <- dist(Z) # the SAME dist() call, on the rescaled table
m_std <- as.matrix(D_std)
sort(m_std["Qatar", ])[2:4]
```

United Arab Emirates	Singapore	Kuwait
1.722698	1.813621	1.854811

Answer. The United Arab Emirates — the answer an atlas would give — then Singapore, then Kuwait.

- `scale(x)` subtracts each column's mean and divides by its standard deviation — its two arguments, `center =` and `scale =`, are both TRUE by default, and today the defaults are exactly what we want.
- **Right:** every column finally gets a vote. One rescaling changed the answer; three formulas did not.
- **The mistake:** it treats the eight columns as unrelated. But `life_exp` and `pop65` move together (correlation 0.66), so a country high on one and low on the other is doing something rare — and a column-by-column rescaling cannot see it.

Question 2.5

Is there a distance that ignores overall size and compares only the shape of two countries' profiles?

```
D_cor <- as.dist(1 - cor(t(X))) # t(X): rows and columns swapped
m_cor <- as.matrix(D_cor)
sort(m_cor["Qatar", ])[2:4]
```

Singapore	Iceland	United States
1.175829e-08	2.346256e-08	3.739270e-08

Answer. Yes — and here it collapses. Read the numbers, not the names: Qatar's nearest neighbours sit at distances of about 10^{-8} .

- Three functions in one line: `t(X)` is the *transpose* — the table with rows and columns swapped — so `cor()` correlates *countries* instead of indicators; `as.dist()` stamps the resulting table as a distance object, so it plays with the same tools as `dist()`'s output.
- **Right:** when the level is noise and the pattern is the signal — lab panels, spending profiles — comparing shapes is exactly what you want.
- **The mistake, twice.** On raw columns every country's profile is one giant income spike, so nearly all 175 profiles have the same shape and nearly every distance is 0 — the method has stopped discriminating. And it throws away the *level*, which here is national income: the thing our question is about.

Question 2.6

One distance in the catalogue claims to handle the scales **and** the columns that move together. What does it say?

The first line below turns the eight columns into uncorrelated columns of equal spread; §3 shows what it does and checks it against `mahalanobis()`. After that line, the straight line as usual:

```
XW <- X %*% solve(chol(cov(X))) # uncorrelated columns, equal spread
D_mah <- dist(XW) # then the straight line, as usual
m_mah <- as.matrix(D_mah)
sort(m_mah["Qatar", ])[2:4]
```

United Arab Emirates	Singapore	Kuwait
2.042716	2.346642	2.557316

Answer. The United Arab Emirates, Singapore, Kuwait — the atlas's answer again, from a distance that also knows how the columns move together.

- **Right:** scales *and* correlations are both handled — the only row of the catalogue that can say that.
- **The mistake:** its answer depends on the other 174 countries, because **S** is computed from the whole table — change the table and every distance moves. The raw distances never do that.

All six on one page

Heat maps make the comparison visible. One call draws one panel — the fixed set below keeps the picture readable (a 175×175 panel is a smear):

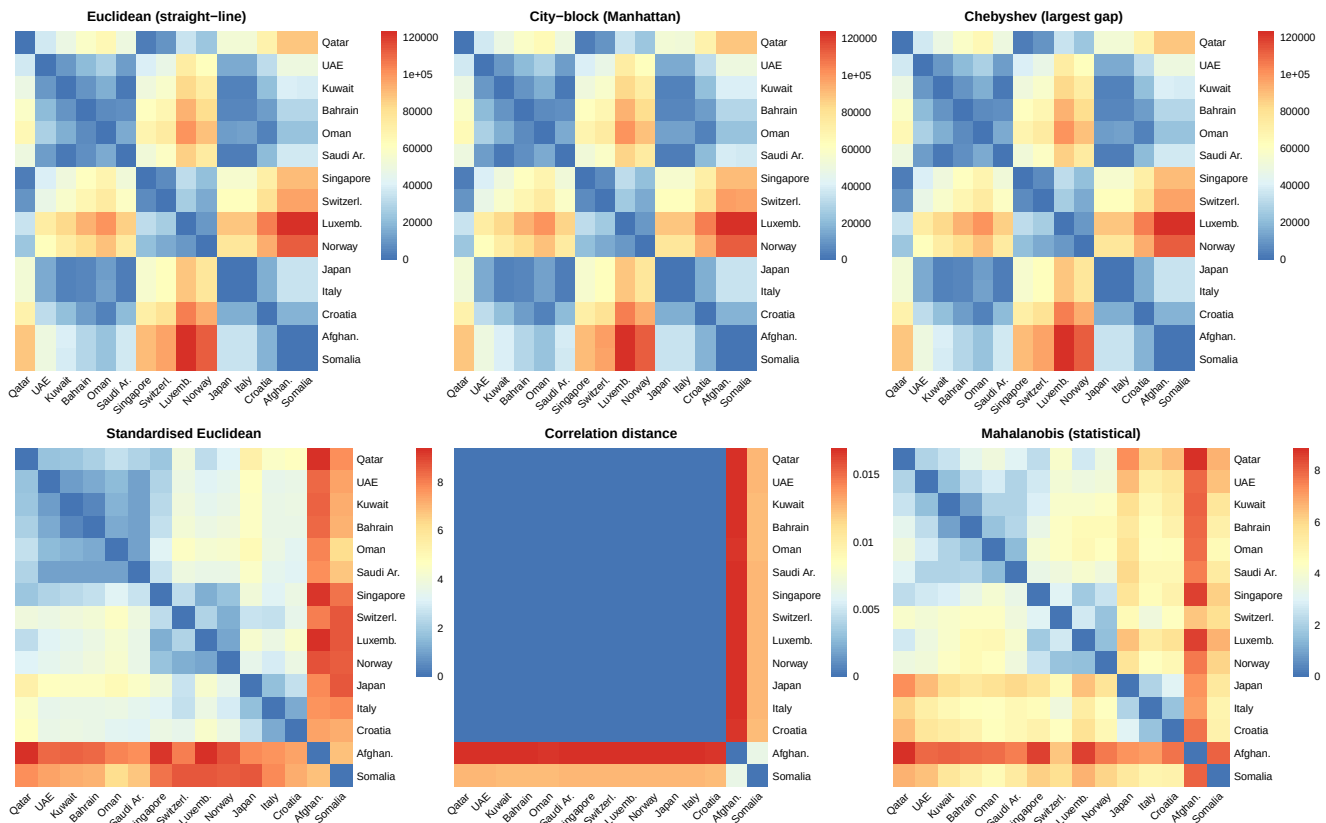
```
sub <- c("Qatar", "United Arab Emirates", "Kuwait", "Bahrain", "Oman",
        "Saudi Arabia", "Singapore", "Switzerland", "Luxembourg",
        "Norway", "Japan", "Italy", "Croatia", "Afghanistan",
        "Somalia, Fed. Rep.") # one fixed order, used by every panel
pheatmap(m_euc[sub, sub], cluster_rows = FALSE, cluster_cols = FALSE)
```

⚠ pheatmap reorders the countries unless you stop it

By default `pheatmap` *clusters* the rows and columns — it would put each panel in its own order, and comparing six differently-ordered panels is meaningless. The two arguments `cluster_rows = FALSE`, `cluster_cols = FALSE` switch that off and keep the one fixed order above in all six panels. The full six-panel figure is drawn by `RProject/R/L5_figures.R`.

? Predict 1 — answer before you read on

Before turning to the figure: **which of the six panels do you expect to be nearly identical to each other?** And what do you expect the correlation-distance panel to look like, given the 10^{-8} values above?



Six distances, one fixed country order (Qatar first). Dark blue: close. Red: far. Each panel has its own colour scale.

How to read this figure

- **One row = one country**; the same 15 countries, in the same order, in all six panels. Qatar is the first row of every panel.
- **Dark blue means close, red means far.** The diagonal is dark blue everywhere: every country is at distance 0 from itself.
- **Each panel has its own colour scale.** The six distances are in different units, so comparing *colours across panels* is meaningless. What is comparable is the **pattern** — who is dark blue beside Qatar in each panel.
- **Top row of panels:** three formulas, one pattern — the income pattern. This is Questions 2.1–2.3 as a picture.
- **Correlation panel:** almost uniformly blue — nearly every pair at distance ≈ 0 — except the two poorest countries in the set. That is the collapse of Question 2.5.
- **Standardised and Mahalanobis panels:** a Gulf block appears beside Qatar, and Afghanistan and Somalia stand out as far from everyone. These two panels resemble each other — *how* they differ is §3.

3 Euclidean against Mahalanobis: same or different?

In this section

The question. Are the Euclidean and the Mahalanobis distance the same thing or different things — and when can they give the same answers?

What we do about it. Run both and compare. Open Mahalanobis up — it is the straight line *after a transformation* — and meet the other two transformed versions on the way. Then compute the exact condition under which the two coincide. Every claim is checked on screen. **What the**

In this section (continued)

section leaves behind: one sentence — the choice between distances is the choice of *what to throw away*.

Question 3.1

Same or different? One line settles it.

```
c(euclidean = names(sort(m_euc["Qatar", ])[2],
mahalanobis = names(sort(m_mah["Qatar", ])[2]))
```

euclidean	mahalanobis
"Singapore"	"United Arab Emirates"

Answer. Different. Same table, same question, two different neighbours — so they are not one method written two ways. The next three questions find out *what* the difference is.

Question 3.2

*What **is** the Mahalanobis distance, then? What did §2's whitening line `X %*% solve(chol(cov(X)))` actually build? R has a function that computes the Mahalanobis distance directly; put the two side by side.*

```
c(whitened_squared = m_mah["Qatar", "United Arab Emirates"]^2,
mahalanobis = mahalanobis(X["Qatar", ], X["United Arab Emirates", ],
cov(X)))
```

whitened_squared	mahalanobis
4.172691	4.172691

Answer. The same number twice: the squared straight-line distance between the two *whitened* rows is the squared Mahalanobis distance d^2 .

- Think of the whitening line as a **picture**, not as algebra: it rescales the columns *and* removes the correlation between them. Afterwards, every direction is on the same footing, and the plain straight line is the right tool again.
- `mahalanobis(x, center, cov)` takes three arguments: the unit(s) `x`, the point measured *from*, and the **covariance matrix** — and it returns the *squared* distance, which is why the left value is squared to match.
- d^2 is **unit-free**: the answer has **no units at all**. Change dollars to any other currency and it does not move.
- (*Ingredients, not syllabus:*) `chol()` factors a covariance matrix; `solve()` undoes a matrix. Both belong to Lecture 6 — today they are one licensed line, checked above.

Question 3.3

And §2's `dist(scale(X))` — where does it sit between the two? Check it against the catalogue's formula for one pair.

```
c(section_2 = m_std["Qatar", "United Arab Emirates"],
recipe = sqrt(sum((X["Qatar", ] - X["United Arab Emirates", ])^2 /
diag(cov(X)))))
```

section_2	recipe
1.722698	1.722698

Answer. Standardised Euclidean is the halfway house: the straight line after each *column* is put on one scale — the correlations survive untouched.

- `diag(cov(X))` picks the diagonal of the covariance matrix — the eight variances — and nothing else. That is the whole difference from Mahalanobis, which uses all of **S**.
- Each gap is counted in standard deviations of its own column, so this distance is **unit-free** too.
- So the ladder is: raw straight line → divide by the spreads (standardised) → also remove the correlations (Mahalanobis).

Question 3.4

Where does correlation distance fit in the same story? With z_i the i -th **row** rescaled to mean 0, sd 1, the claim is $\|z_i - z_j\|^2 = 2(p-1)(1 - r_{ij})$ exactly. Check it.

```
Zr <- t(scale(t(X)))           # scale each ROW -- each country's profile
R  <- cor(t(X))                # correlation between two countries' rows
c(gap_squared = sum((Zr["Qatar", ] - Zr["United Arab Emirates", ])^2),
  formula      = 2 * (8 - 1) * (1 - R["Qatar", "United Arab Emirates"]))
max(abs(as.matrix(dist(Zr))^2 - 2 * (8 - 1) * (1 - R)))
```

```
gap_squared      formula
6.561779e-06 6.561779e-06
[1] 8.578533e-15
```

Answer. Exactly, for every one of the $175 \times 174/2$ pairs at once — the largest disagreement anywhere is 10^{-14} , floating-point dust.

- So $1-r$ is the squared straight line between *row-standardised* profiles, divided by the constant $2(p-1)$.
- The row-standardising throws away each country's *level* and keeps only the *shape* — which is why it collapsed in Question 2.5, where the level is national income.
- (The $p-1=7$ is there because R's `scale()` uses the $n-1$ denominator — the same convention as `sd()`.)

Fact 3.1: One idea, three disguises

All three repairs are the ordinary straight-line distance, computed after a transformation of the table. The only difference between them is *which* transformation — that is, **what each one decided to throw away**:

- **standardised Euclidean** throws away the *scales* of the columns;
- **Mahalanobis** throws away the scales *and* the correlations between columns;
- **correlation distance** throws away the *level* of each row and keeps only its shape.

When can Euclidean and Mahalanobis be the same?

? Predict 2 — answer before you read on

Whitening rescales the columns and removes their correlations. **State the exact condition under which it would have nothing to do — so that the Euclidean and the Mahalanobis distances come out equal.** Commit to a sentence before reading on.

Build a tiny table where the condition holds exactly, and check:

```
U <- cbind(a = c(1, 2, 3, 4), b = c(1, -1, -1, 1))
Z0 <- scale(U) # mean 0, sd 1 -- and cor(a, b) is exactly 0
round(cov(Z0), 10) # the covariance matrix is the identity
max(abs(dist(Z0) - dist(Z0 %*% solve(chol(cov(Z0))))))
```

```
a b
a 1 0
b 0 1
[1] 0
```

Now put correlation back in — replace `b` by `a + b` — and ask again; then ask on the real table:

```
V <- U; V[, "b"] <- U[, "a"] + U[, "b"] # put correlation back in
cor(V)[1, 2]
max(abs(dist(scale(V)) - dist(V %*% solve(chol(cov(V))))))
max(abs(D_std - D_mah)) # and on the real 175 countries
```

```
[1] 0.745356
[1] 0.9312748
[1] 4.159764
```

Fact 3.2: When the two are the same, settled

- **Euclidean and Mahalanobis return the same distance matrix exactly when the covariance matrix is the identity** — every column with variance 1, every pair of columns uncorrelated. On `Z0` that holds, and the difference prints as exactly 0: the whitening had nothing left to do.
- **After standardising, only the correlations remain in play:** with correlation 0.75 put back into the toy table, the standardised and Mahalanobis matrices separate (0.93 at the widest).
- On the real table the widest gap is **4.16** — the correlations among the eight indicators do real work, and the two methods are genuinely different answers.

4 A workflow for a distance problem

In this section

The question. *A new table lands on your desk and the question is about similarity. What do you actually do, in what order?*

What we do about it. Four steps, each illustrated on today's table, ending in a one-page decision table. **What the section leaves behind:** the recipe you will use on Worksheet 5, on the home problems, and in every later lecture that starts from a distance.

Step 1 — look at the columns before computing anything

Question 4.1

What units are the columns in, and how much does each one vary?

```
round(apply(X, 2, sd), 2) # workflow step 1: every column's spread
```

life_exp	infant_mort	fertility	pop65	urban_pct
8.20	22.75	1.26	7.14	22.08
gdp_pc	health_exp_pct	internet_pct		
28313.17	3.27	25.47		

Answer. Mixed units, and spreads four orders of magnitude apart.

- `apply(X, 2, sd)` applies `sd()` to dimension 2 of `X` — its columns; 1 would mean the rows.
- One read of this line tells you the raw straight line is doomed here: `gdp_pc` will out-vote everything by a factor of thousands. This is the single cheapest check in the workflow.

Step 2 — columns in one unit: the plain distances are fine

Question 4.2

Keep only the four columns measured in percent. Who is nearest Qatar?

```
Xp <- X[, c("pop65", "urban_pct", "health_exp_pct", "internet_pct")]
mp <- as.matrix(dist(Xp))      # four columns, all in percent
sort(mp["Qatar", ])[2:4]
```

```
      Kuwait      Bahrain Singapore
3.063845    3.430987 11.713480
```

Answer. Kuwait and Bahrain, close; Singapore, further. A sensible answer with no repair at all — like §1's P2.

- When the columns share a unit and vary by comparable amounts, the raw straight line (or city-block, or largest-gap — pick by purpose) is legitimate as it stands.
- The mistake to avoid here is the opposite one: standardising when nothing needed repair changes the answer for no reason.

Step 3 — mixed units: standardise, then measure

No new computation needed — this is Question 2.4: `dist(scale(X))` gave **the United Arab Emirates, Singapore, Kuwait**. When the columns are in different units, divide each by its own spread first; the distance becomes unit-free and every column gets a vote.

Step 4 — hunting unusual units when columns move together

When the purpose is “*which units are unusual?*” and the columns are correlated, measure each unit from the **centre** of the table with the Mahalanobis distance — it alone flags rare *combinations*. One country shows why the extra step matters:

? Predict 3 — answer before you read on

Somewhere in this data is a country whose income is almost exactly the world average, on a pair of columns where income is the loud one. **Where will each of the two distances place it — ordinary or unusual?**

```
P <- X[, c("gdp_pc", "pop65")]      # two columns, dollars + percent
sel <- c("Qatar", "Singapore", "United Arab Emirates", "Croatia")
round(cbind(P[sel, ], d2 = mahalanobis(P, colMeans(P), cov(P))[sel]), 2)
mean(P[, "gdp_pc"])                # the world average income
```

```
      gdp_pc pop65  d2
Qatar    88701.46  1.39 16.78
Singapore 91227.70 12.69  8.47
United Arab Emirates 50759.76 1.64  6.31
Croatia   18465.53 22.48  4.88
[1] 18450.92
```

Croatia earns \$18 465.53. The world average is \$18 450.92. The gap is \$14.61. Now rank all 175 countries by each method's distance from the centre:

```
d2p <- mahalanobis(P, colMeans(P), cov(P)) # from the centre, all 175
eup <- sqrt(rowSums(sweep(P, 2, colMeans(P))^2))
c(straight_line = rank(-eup)[["Croatia"]], by_d2 = rank(-d2p)[["Croatia"]])
```

```
straight_line      by_d2
      175           14
```

Fact 4.1: Croatia — 175th of 175, and 14th of 175

- Three functions in the chunk: `sweep(P, 2, colMeans(P))` subtracts the column means from every row; `rowSums()` adds across each row; `rank(-x)` ranks largest-first, and `[["Croatia"]]` pulls one country's rank by name.
- **By straight-line distance from the centre, Croatia is the single most ordinary country in the world** — rank 175 of 175 — because \$14.61 of income is all that distance can see.
- **By d^2 it is the 14th most unusual**: 22.5% of Croatians are over 65 — 1.78 standard deviations above the world — *at an income where that does not happen*. The two columns move together (0.59); Croatia moves against the grain.
- **161 places apart, on one country, from one file**. That is what the extra transformation buys, and what the raw distance can never see.

The decision table

Your purpose	Reach for	The R
values close, columns already in one unit	Euclidean	<code>dist(X)</code>
one wild column must not dominate by squaring	city-block	<code>dist(X, "manhattan")</code>
the single worst gap is the question	Chebyshev	<code>dist(X, "maximum")</code>
mixed units, columns roughly unrelated	standardised Euclidean	<code>dist(scale(X))</code>
rare <i>combinations</i> ; columns move together	Mahalanobis	<code>mahalanobis()</code> , or <code>dist()</code> on the whitened table
the <i>shape</i> of a profile, not its level	correlation distance	<code>as.dist(1 - cor(t(X)))</code>

And always step 1 first: `apply(X, 2, sd)` — ten seconds that decide the whole left column.

5 Warm-up

In this section

The question. *Did any of that stick?*

What we do about it. Five one-line questions, out loud, notes closed. The in-class worksheet follows in the same room.

Warm-up — say these aloud, nothing to write

1. What does a distance of *zero* between two countries mean?

Warm-up (continued)

2. Name one distance that called **Singapore** Qatar's nearest neighbour, and one that called it **the United Arab Emirates**.
3. In a word or two each: what does standardised Euclidean throw away? Mahalanobis? Correlation distance?
4. A new table lands on your desk. What is the *first* line of R the workflow tells you to run, before any distance?
5. Two heat-map panels, two colour scales: what may you compare across them — the colours, or the pattern?

6 Problems to try at home — all in R

In this section

The question. *Does anything from today change on a wider table?*

What we do about it. Three problems, all on `whwide2022` — the same 175 countries, 52 indicators. Each asks you to run something and write **two sentences** about what came back. Short answers are at the end of the section; full solutions in the practice session.

The wide file holds 52 indicators, but only **23 of them are complete for all 175 countries** — the rest have gaps. The problems below use those 23, so nothing drops out (`w` was read in §1):

```
ind <- w[, 5:56] # the 52 indicators of whwide2022
keep <- colSums(is.na(ind)) == 0 # complete for all 175 countries
W <- as.matrix(ind[, keep]); rownames(W) <- w$country
dim(W)
```

```
[1] 175 23
```

Home Exercise 1. The straight line, wider. On `W`, compute Qatar's three nearest neighbours under the raw straight-line distance (`dist(W)`) and under the standardised straight-line distance (`dist(scale(W))`). Compare both lists with the eight-column lists from §2, and write two sentences: *what changed, and which column is responsible?* (`sort(as.matrix(...)[ "Qatar", ])[2:4]` reads the answer off, as in §2.)

Home Exercise 2. Two rankings, priced. Use exactly these twelve columns — a set checked in advance so that the statistical distance can be computed (with many more columns than this, R stops with an error; understanding why is Lecture 6):

```
v12 <- c(colnames(X), "mobile", "gdp_growth", "death_rate", "birth_rate")
W12 <- W[, v12] # a set checked to be safe for d^2
```

Compute every country's distance from **Qatar** under standardised Euclidean (`dist(scale(W12))`) and under Mahalanobis (`dist(W12 %>% solve(chol(cov(W12))))`). Report each method's three nearest neighbours and the Spearman rank correlation between the two full rankings — `cor(a, b, method = "spearman")` correlates the two *orderings* rather than the two value lists. Two sentences: *where do the two methods agree, and what single ingredient explains the gap?*

Home Exercise 3. Shape, not level. On `W`, use correlation distance (`as.dist(1 - cor(t(W)))`) to find the country whose *profile shape* is closest to Qatar's. Two sentences: *who is it, and why is this not the same question as Problem 1?*

Short answers. 1) Raw: Gambia, The / Moldova / Lithuania — the loudest column is now total population (`pop_tot`), so the straight line matches Qatar with countries of similar population size and

forgets income entirely; standardised: United Arab Emirates / Kuwait / Bahrain — Singapore drops out of the top three. 2) Standardised: Kuwait, the United Arab Emirates, Singapore; Mahalanobis: Kuwait, the United Arab Emirates, Oman; Spearman 0.674. They agree at the very top and part company below it; the only ingredient one has and the other lacks is the correlations between the twelve columns. 3) Singapore, at a correlation distance of about 2×10^{-4} — Problem 1 asks who matches Qatar’s values; this asks who matches the shape of its profile, sizes ignored — a different question, and on raw mixed-unit columns a nearly degenerate one, as in §2.

7 FAQ

In this section

The question. *The questions this lecture usually leaves behind — answered in two lines each.*

What we do about it. Read these after the worksheet; each answer points back to the section that earned it.

FAQ — Lecture 5

1. Is one of the six distances the “correct” one?

No. Each is correct *for a purpose* — that is what §4’s decision table is for. The wrong move is not picking a “bad” distance; it is not being able to say *why* you picked yours.

2. Do I report d or d^2 ?

Either, as long as you say which. `mahalanobis()` returns the *squared* distance; a square root never changes an order, so rankings are identical both ways.

3. Does `dist()` lose the country names?

No — they ride along, and `as.matrix()` shows them as row and column names. If your names are missing, they were never on the matrix: set `rownames()` before calling `dist()`.

4. Why not just always standardise, to be safe?

Two reasons. If the columns already share a unit, standardising changes the answer for no reason (§4, step 2). And standardising fixes only the *scales* — it cannot see columns that move together, which is Mahalanobis’s job (§3).

5. Is a bigger d^2 always “more unusual”?

It *ranks*. How large is large enough to call a unit an exception needs a probability model, and Stage 1 has none — that is Lecture 11.

6. Can the Mahalanobis distance always be computed?

No — it needs $\mathbf{S}^{-1}$, and for some tables R stops with an error instead. That is why Home Problem 2 fixes its twelve columns in advance. When an inverse exists, and how to know before asking R, is Lecture 6.

7. When do Euclidean and Mahalanobis agree?

Exactly when the covariance matrix is the identity (§3): columns uncorrelated, each with variance 1 — then whitening has nothing to do. On real mixed-unit tables, they differ.

8. Do I standardise before or after computing distances?

Before: `scale()` the table, then `dist()` it. A distance matrix has no columns left to standardise.

Wrap-up

Summary of Lecture 5

- **A distance is one number per pair of countries**, obeying four rules — and the straight-line distance *provably* obeys all four (§1), with Cauchy–Schwarz closing the triangle inequality.
- **One column, then two in the same unit: the straight line is honest.** The UAE was Qatar’s neighbour both times, Monaco the farthest. **One column in dollars broke it:** Singapore replaced the UAE because the age columns stopped voting.
- **On the full table the formula does not matter and the answer is wrong anyway.** Straight-line, city-block and largest-gap all returned Singapore — income decides alone.
- **Making the columns comparable is the decision that moves the answer.** One `scale()` turned Singapore into the United Arab Emirates; three formulas moved nothing.
- **Euclidean and Mahalanobis are different** — and Mahalanobis is the straight line *after whitening*: rescale the columns and remove their correlations, checked against `mahalanobis()` to the last digit. **They coincide exactly when the covariance matrix is the identity.**
- **Standardised Euclidean and d^2 are unit-free: the answer has no units at all.** Correlation distance goes further and ignores the level itself — the wrong tool here, where the level is national income.
- **Heat maps: one fixed country order, one colour scale per panel.** Compare patterns across panels, never colours.
- **The workflow (§4):** look at the spreads first; same unit → plain distances; mixed units → standardise; rare combinations → Mahalanobis (Croatia: 175th of 175 by one verdict, 14th by the other); shape → correlation distance.
- **The R to remember:** `dist()` and its method = argument; `scale()`; `apply(X, 2, sd)`; the whitening line checked against `mahalanobis()`; `1 - cor(t(X))`; `pheatmap()` with the clustering switched off.

Next lecture (Sun, Sep 13): *Lecture 6 — matrix algebra.*

Quiz 1 scope — read both halves.

Included (L1–L5): the $n \times p$ data array and its dimensions; reading `dim` / `str` / `head` output; $\bar{x}$, **S**, **R** by hand and in R; the $n - 1$ convention; reading a correlation matrix or `corrplot`; choosing a display for a question; what a distance is and the four rules; the six distances — names, formulas, worked values; reading `dist()` output and a heat map; what standardising fixes and what it cannot see; when Euclidean and Mahalanobis agree; the workflow table — which distance for which purpose; `mahalanobis()` as the check on the whitening line.

Not included: the proof of the triangle inequality (read it, don’t memorise it); eigenvalues and eigenvectors (L7); matrix algebra, inverses and when they exist (L6); the multivariate normal and χ^2 cut-offs (L11–L13); *populations* rather than samples (L9); clustering (L26).

Format: bubble sheet, 20 items, 50 minutes. **You do not run R during the quiz** — every piece of output you need is on the paper.